Creando un documento de diseño

DPOO - Lindsay Pinto

LO QUE PIDE EL PROYECTO

A

DIAGRAMA DE CLASES DE DISEÑO

Todas las clases con relaciones, atributos y métodos.

B

DIAGRAMA DE ALTO NIVEL

Todas las clases y relaciones, sin todo el detalle. Sirve para entender la estructura.

C

DIAGRAMAS DE SECUENCIA

Para las funcionalidades críticas, diciendo por qué lo son.

SI SOLO ENTREGAN A, B Y C → INSUFICIENTE

Además deben mostrar:

- Contexto, alcance y no-objetivos
- Componentes y cómo se partió el problema
- Roles, estereotipos y responsabilidades
- Estilo de control y por qué
- Decisiones clave con alternativas y trade-offs
- Persistencia y preocupaciones transversales
- Coherencia entre diagramas y código

Las actividades 2 y 3 piden que el código sea coherente con el modelo de clases y las secuencias.

DISEÑAR ES TOMAR DECISIONES

Cada decisión de diseño tiene cuatro partes, y el documento las registra:

PROBLEMA	+	ALTERNATIVAS	+	ELECCIÓN	+	TRADE-OFF
¿Qué hay que resolver?		¿Qué opciones había?		¿Cuál tomamos?		¿Qué ganamos y qué sacrificamos?

- No existe una solución única ni óptima: existe la que mejor equilibra RF, RNF y restricciones.
- El documento de diseño no es un manual de implementación: explica por qué el sistema es así.
- El proceso es iterativo: volver atrás y corregir una decisión es normal.

PASOS IMPORTANTES

01
ENTENDER EL PROBLEMA

02
DESCOMPONER

03
CANDIDATOS Y ESTEREOTIPOS

04
RESPONSABILIDADES

05
COLABORACIONES

06
CLASES

07
DECISIONES

08
VALIDAR

↺ **Es iterativo: el paso 6 puede obligarlos a ajustar los pasos 3 a 5.**

EL CASO: UNA CAFETERÍA

ENUNCIADO (RESUMIDO)

- Los cajeros registran pedidos para recoger en una aplicación de consola en Java.
- El menú (productos, combos e ingredientes con su precio) se carga desde archivos de texto.
- Un producto se puede ajustar: agregar ingredientes (con costo) o quitarlos.
- Un combo agrupa productos con un descuento.
- Al cerrar el pedido se genera la factura (subtotal, impuesto de 8 % y total) y se guarda en un archivo.
- Se puede consultar un pedido anterior por su número.

PEDIDO #12

Cliente: Ana

Capuchino	6.000
+ shot extra	1.500
Combo desayuno	15.300
americano + sándwich	−10 %
Subtotal	22.800
Impuesto 8 %	1.824
TOTAL	**24.624**

La factura que el sistema debe generar

01

LOS 8 PASOS

ENTENDER EL PROBLEMA

QUÉ HACER

1. Releer el enunciado y listar los requerimientos funcionales (qué debe hacer el sistema).
2. Listar los no funcionales: facilidad de cambio, persistencia, usabilidad...
3. Anotar las restricciones: lenguaje, interfaz, archivos o base de datos, tiempo.
4. Escribir objetivos y no-objetivos: lo que queda explícitamente por fuera.

PRODUCE EN EL DOCUMENTO

Sec. 1 · Contexto y alcance

- Descripción breve del problema
- RF, RNF y restricciones
- Objetivos y no-objetivos

PREGUNTA DE CONTROL

¿Alguien que no leyó el enunciado entiende qué resuelve el sistema y qué queda por fuera?

CONTEXTO Y ALCANCE

REQUERIMIENTOS FUNCIONALES

- R1 · Mostrar el menú
- R2 · Iniciar un pedido con el nombre del cliente
- R3 · Agregar productos, ajustados o combos
- R4 · Cerrar el pedido y guardar la factura
- R5 · Consultar un pedido por número

NO FUNCIONALES Y RESTRICCIONES

- Java, aplicación de consola
- El menú vive en archivos de texto
- Agregar un producto al menú no debe exigir cambiar código
- Un solo pedido en curso a la vez

NO-OBJETIVOS

- Pagos con tarjeta o en línea
- Inventario de ingredientes
- Usuarios y contraseñas
- Domicilios

Los no-objetivos evitan trabajo inútil: nadie diseña un módulo de pagos que no se pidió.

DESCOMPONER EN COMPONENTES

QUÉ HACER

1. Dibujar el sistema como una caja negra con lo que está afuera: usuarios, archivos, otros sistemas.
2. Partirlo en 3 a 5 componentes gruesos, cada uno descrito en una frase.
3. Separar lo especializado o complejo (interfaz, persistencia) para tratarlo aparte.
4. Definir la frontera de cada componente: qué hace y cómo se ve desde afuera.

PRODUCE EN EL DOCUMENTO

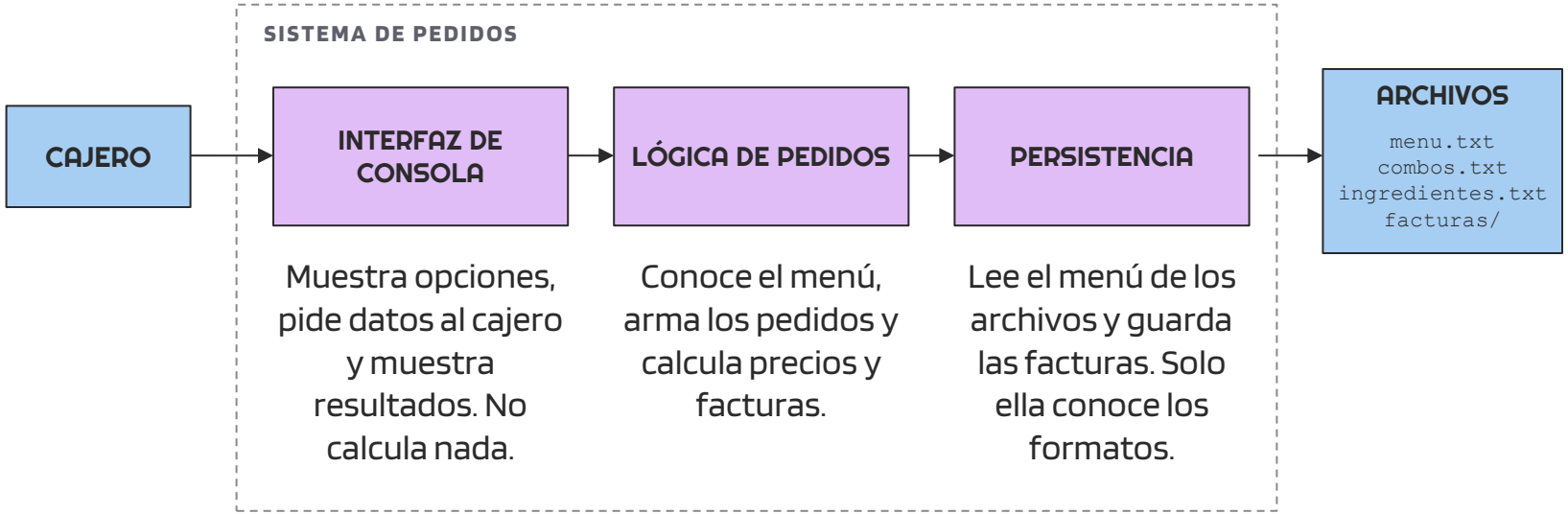
Sec. 2 · Visión general

- Diagrama de contexto
- Diagrama de componentes
- Por qué se partió así

PREGUNTA DE CONTROL

¿Cada componente se puede explicar en una frase, sin usar "y además..."?

COMPONENTES DEL SISTEMA



Por qué esta división: si mañana la interfaz es gráfica, solo cambia la interfaz; si el menú pasa a una base de datos, solo cambia la persistencia. Los cálculos no se tocan.

CANDIDATOS Y ESTEREOTIPOS

QUÉ HACER

1. Buscar candidatos desde varias miradas: el dominio, lo que hace el sistema, lo que fluye por él y los agrupadores.
2. Asignar a cada candidato un estereotipo.
3. Escribir un buen nombre y una descripción de una línea.
4. Descartar, fusionar o agregar: la lista inicial no es definitiva.

PRODUCE EN EL DOCUMENTO

Sec. 3 · Roles y candidatos

- Tabla: candidato, descripción, estereotipo
- Lo que se descartó y por qué

PREGUNTA DE CONTROL

¿Hay nombres vagos como Gestor, Manager o Datos? ¿Se entiende el rol solo con el nombre?

¿SABE, HACE O DECIDE?

SABE guarda información	**HACE** trabaja para otros	**DECIDE** controla el flujo
«INFORMATION HOLDER» Guarda y entrega información sobre sí mismo. **Clase:** Ingrediente, ProductoMenu	**«SERVICE PROVIDER»** Hace un trabajo especializado cuando se lo piden. **Clase:** un CalculadorDescuentos	**«CONTROLLER»** Toma decisiones con reglas y dice a otros qué hacer. **Clase:** ConsolaCafeteria
«STRUCTURER» Organiza a otros objetos y sabe cómo se relacionan. **Clase:** Pedido, Combo	**«INTERFACER»** Traduce entre dos mundos: usuario, archivos, BD. **Clase:** CargadorMenu, RegistroPedidos	**«COORDINATOR»** Reparte el trabajo a otros sin decidir mucho. **Clase:** Cafeteria
¿Sabe de sí mismo o organiza a otros?	**¿Hace un trabajo o pasa algo de un lado a otro?**	**Si le quito las decisiones, ¿queda vacío? → controller**

EJEMPLO · CAFETERÍA · PASO 03

CANDIDATOS DE LA CAFETERÍA

Candidato	Descripción (una línea)	Estereotipo
Cafeteria	Conoce menú y pedidos; coordina las solicitudes	**«coordinator»**
ConsolaCafeteria	Lee la opción del cajero y decide qué pedir	**«controller»**
Pedido	Agrupa los productos y calcula sus totales	**«structurer»**
ProductoMenu	Producto del menú con nombre y precio base	**«information holder»**
ProductoAjustado	Producto base con ingredientes de más o de menos	**«structurer»**
Combo	Grupo de productos con un descuento	**«structurer»**
Ingrediente	Ingrediente con su costo adicional	**«information holder»**
CargadorMenu	Lee los archivos y crea los objetos del menú	**«interfacer»**
RegistroPedidos	Guarda y busca facturas en archivos	**«interfacer»**

DESCARTADOS

- Factura → no es clase: es el texto que genera Pedido.
- Cliente → solo es un nombre: atributo de Pedido.
- Menú → lo mantiene Cafeteria.

Decidir qué NO es una clase también es una decisión de diseño.

ASIGNAR RESPONSABILIDADES

QUÉ HACER

1. Listar responsabilidades de tres tipos: saber, hacer y decidir.
2. Buscarlas en los requerimientos, en lo implícito, en "si pasa… entonces…" y en el ciclo de vida.
3. Asignar cada una a un candidato, empezando por las más gruesas.
4. Usar verbos precisos (no "manejar" ni "gestionar"); sin duplicados ni huérfanas.

PRODUCE EN EL DOCUMENTO

Sec. 4 · Responsabilidades

- Tabla: responsabilidad, tipo, asignada a
- Justificación de las difíciles

PREGUNTA DE CONTROL

¿Alguna clase acumula cosas sin relación entre sí? ¿Alguna responsabilidad quedó sin dueño?

RESPONSABILIDADES

#	Responsabilidad	Tipo	Asignada a
1	Saber el menú (productos, combos, ingredientes)	Saber	**Cafeteria**
2	Leer el menú desde los archivos	Hacer	**CargadorMenu**
3	Crear un pedido y asignarle número	Hacer	**Cafeteria**
4	Decidir si se puede iniciar o cerrar un pedido	Decidir	**Cafeteria**
5	Agregar un producto al pedido	Hacer	**Pedido**
6	Calcular el precio de un producto ajustado	Hacer	**ProductoAjustado**
7	Calcular el precio de un combo con descuento	Hacer	**Combo**
8	Calcular subtotal, impuesto y total	Hacer	**Pedido**
9	Generar el texto de la factura	Hacer	**Pedido**
10	Guardar y buscar facturas en archivos	Hacer	**RegistroPedidos**
11	Pedir datos al cajero y mostrar resultados	Hacer	**ConsolaCafeteria**

POR QUÉ ASÍ

- Cada producto calcula su propio precio: la información está ahí.
- Pedido suma sin preguntar el tipo de cada producto.
- Ninguna clase del modelo sabe de archivos.
- Quien crea los pedidos también les da el número.

COLABORACIONES Y CONTROL

QUÉ HACER

1. Elegir 3 o 4 escenarios críticos: los requerimientos más importantes o complejos.
2. Para cada uno: quién recibe la solicitud, quién decide y a quién delega.
3. Dibujar el diagrama de secuencia de cada escenario.
4. Identificar el estilo de control (centralizado, delegado o disperso) y justificarlo.

Prefieran pocas interacciones significativas a muchas pequeñas. Diseñen primero la funcionalidad más importante.

PRODUCE EN EL DOCUMENTO

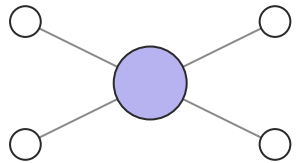
Sec. 5 · Colaboraciones

- Diagramas de secuencia (c)
- Por qué esos escenarios
- Estilo de control justificado

PREGUNTA DE CONTROL

¿Muchas conexiones fuera del "vecindario", muchos mensajes pequeños o muchos caminos alternativos?

ESTILOS DE CONTROL

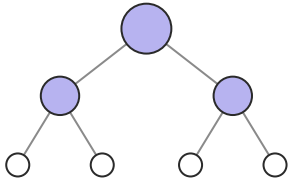


CENTRALIZADO

Pocas clases (controladores) toman casi todas las decisiones.

+ Fácil encontrar dónde se decide.

– Controladores complejos y acoplados; el resto queda vacío.

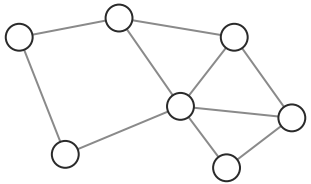


DELEGADO

Coordinadores reparten trabajo; cada parte decide lo suyo.

+ Menor acoplamiento; cambios localizados.

– Puede haber colaboraciones complejas.



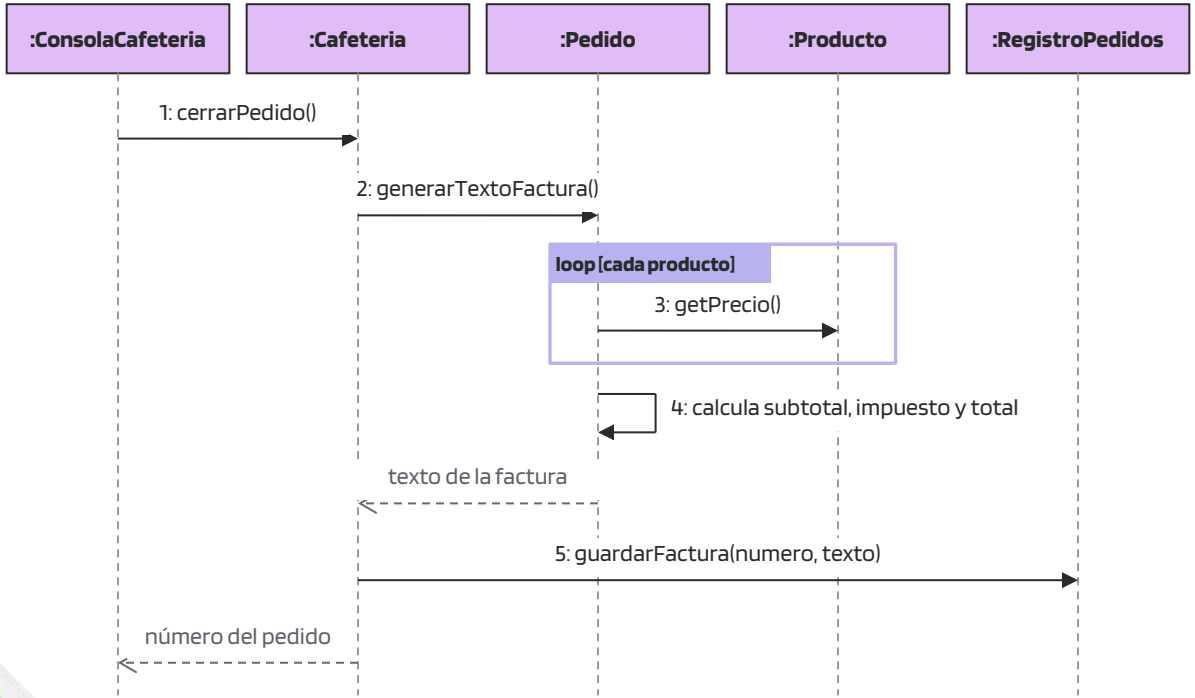
DISPERSO

Las decisiones se reparten en muchos objetos pequeños.

+ Cada decisión es pequeña.

– Difícil encontrar el control; un cambio afecta muchas clases.

SECUENCIA: CERRAR UN PEDIDO



ESCENARIOS CRÍTICOS

1. Cargar el menú
2. Agregar un producto ajustado
3. Cerrar el pedido (este)

Es crítico: combina cálculo, polimorfismo y archivos.

CONTROL: DELEGADO

Cafeteria coordina pero no calcula: cada producto sabe su precio y Pedido arma la factura.

ATERRIZAR A CLASES

QUÉ HACER

1. Convertir cada candidato en clase o interfaz, o dejarlo para otra iteración.
2. Definir atributos y estructuras de datos para las relaciones (listas, mapas…).
3. Especificar contratos: métodos con nombre, parámetros y retorno.
4. Decidir relaciones: asociación, herencia, interfaces y cardinalidades.

PRODUCE EN EL DOCUMENTO

Sec. 6–7 · Modelo de clases

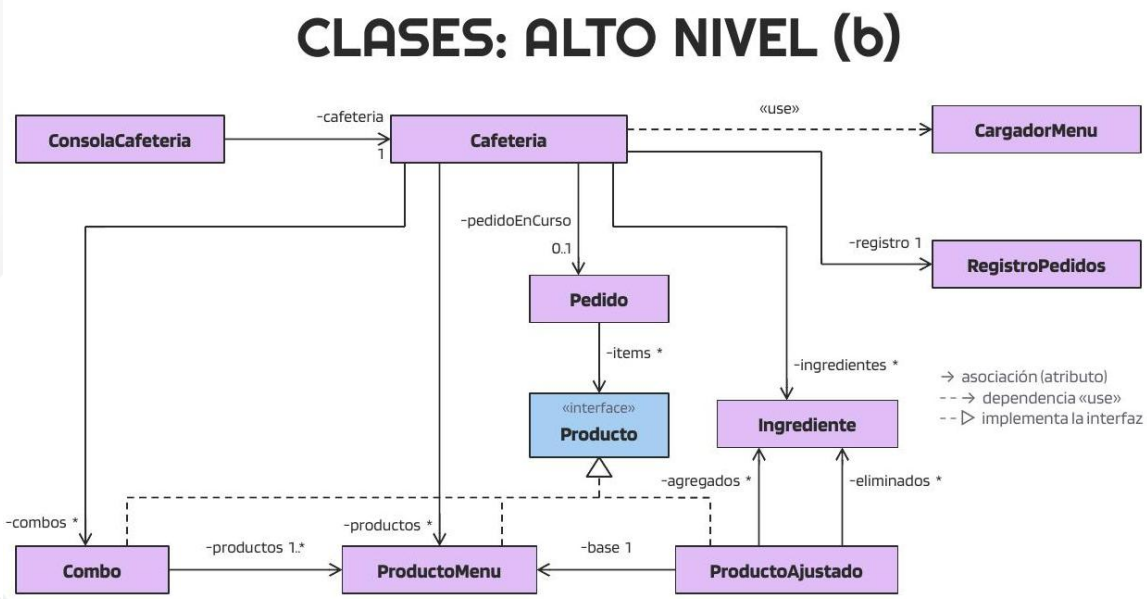
- Diagrama de alto nivel (b)
- Diagrama detallado (a)
- Descripción de cada clase

PREGUNTA DE CONTROL

¿Cada mensaje de los diagramas de secuencia existe como método en el diagrama de clases?

ALTO NIVEL (b)

MODELO DE CLASES



DETALLADO (a) · FRAGMENTO

PEDIDO

```
- numero: int
- nombreCliente: String
- items: List<Producto>
- IMPUESTO: double = 0.08
```

```
+ agregarProducto(p: Producto)
+ getPrecioNeto(): int
+ getImpuesto(): int
+ getPrecioTotal(): int
+ generarTextoFactura(): String
```

REGISTRAR DECISIONES

QUÉ HACER

1. Elegir las decisiones importantes (no todas): estructura, herencia o composición, persistencia, dónde está el control.
2. Para cada una: problema, alternativas consideradas y opción elegida.
3. Escribir lo que se gana y lo que se sacrifica (el trade-off).
4. Relacionarla con un requerimiento o una restricción.

PRODUCE EN EL DOCUMENTO

Sec. 8 · Decisiones de diseño

- Una ficha por decisión
- Alternativas descartadas y por qué

PREGUNTA DE CONTROL

¿Podemos decir qué sacrificamos con cada decisión? "Porque es más fácil" no es una justificación.

Es la sección que más pesa: el enunciado pide hacer énfasis en justificar las decisiones clave.

DOS DECISIONES CLAVE

D1 · ¿CÓMO TRATAR IGUAL A PRODUCTOS SIMPLES, AJUSTADOS Y COMBOS?

OPCIONES	A) Una clase Producto con un atributo "tipo" y muchos if. B) Interfaz Producto con tres implementaciones.
ELEGIDA	B
GANAMOS	Pedido suma precios sin preguntar el tipo; una bebida con tamaño sería una clase nueva sin tocar Pedido.
SACRIFICAMOS	Más clases que entender; Combo y ProductoAjustado dependen de ProductoMenu.

D2 · ¿QUIÉN CALCULA EL TOTAL Y ARMA LA FACTURA?

OPCIONES	A) Cafeteria hace todos los cálculos (centralizado). B) Cada producto calcula su precio y Pedido sus totales (delegado).
ELEGIDA	B
GANAMOS	El cálculo queda junto a la información; Cafeteria es un coordinador simple.
SACRIFICAMOS	Para seguir un total hay que recorrer varias clases (por eso hay diagrama de secuencia).

VALIDAR Y MANTENER COHERENCIA

QUÉ HACER

1. Revisar las 4 recomendaciones: reducir complejidad, pensar en cambios, reducir acoplamiento, preferir inyección de dependencias.
2. Revisar coherencia: cada clase hace cosas relacionadas y trabaja en un solo dominio.
3. Documentar preocupaciones transversales: persistencia, errores, validaciones.
4. Al implementar: si el código cambia, actualizar diagramas y decisiones.

PRODUCE EN EL DOCUMENTO

Sec. 9 · Riesgos y consistencia

- Persistencia y manejo de errores
- Cambios previstos
- Documento coherente con el código

PREGUNTA DE CONTROL

Si ponemos el código al lado de los diagramas, ¿coinciden nombres de clases, métodos y relaciones?

VALIDACIÓN DEL DISEÑO

Recomendación	Cómo la cumple la cafetería
Reducir la complejidad	Cada clase se explica en una frase; ninguna hace todo.
Pensar en los cambios	Bebida con tamaño → nueva clase que implementa Producto. Base de datos → solo cambian CargadorMenu y RegistroPedidos.
Reducir el acoplamiento	Pedido solo conoce la interfaz Producto, no sus implementaciones.
Inyección de dependencias	Cafeteria recibe CargadorMenu y RegistroPedidos en su constructor.

DIAGRAMAS VS. CÓDIGO

✓ guardarFactura(numero, texto) = void guardarFactura(int, String)

✓ getPrecio() = int getPrecio() en Producto

✗ apareció imprimirFactura() en la consola → agregarlo al diagrama

PREOCUPACIONES TRANSVERSALES

- Si falta un archivo del menú: mensaje claro, sin que la aplicación se cierre.
- No se puede cerrar un pedido vacío.

02

PLANTILLAS

ESTRUCTURA DEL DOCUMENTO

#	Sección	Contenido mínimo	Paso
1	**Contexto y alcance**	Problema, RF, RNF, restricciones, objetivos y no-objetivos	1
2	**Visión general**	Diagrama de contexto y de componentes, con explicación	2
3	**Roles y candidatos**	Tabla de candidatos con descripción y estereotipo	3
4	**Responsabilidades**	Tabla de responsabilidades asignadas y justificación	4
5	**Colaboraciones (c)**	Secuencias de escenarios críticos + estilo de control	5
6	**Clases: alto nivel (b)**	Todas las clases y relaciones, sin detalle	6
7	**Clases: detallado (a)**	Atributos, métodos con firmas, relaciones; descripción por clase	6
8	**Decisiones de diseño**	Fichas de decisión con alternativas y trade-offs	7
9	**Riesgos y consistencia**	Persistencia, errores, cambios previstos, coherencia con el código	8

(a), (b), (c) = los tres diagramas que exige el enunciado.

FICHA DE DECISIÓN

Decisión	¿Quién lee el menú de los archivos y guarda las facturas?
Alternativas	A) Cada clase lee o escribe su parte (ProductoMenu lee su línea, Pedido escribe su factura). B) Dos clases dedicadas: CargadorMenu y RegistroPedidos.
Elegida	B · Persistencia separada del modelo.
Ganamos	El modelo no sabe nada de archivos; si el formato cambia o se pasa a una base de datos, solo cambian esas dos clases.
Sacrificamos	Dos clases más, y Cafeteria debe pasarles los objetos que se leen o se guardan.
Relacionada con	Restricción: el menú vive en archivos de texto · RNF: facilidad de cambio.

Ejemplo de la cafetería. Recomendación: entre 3 y 6 fichas para las decisiones que más cambiaron la estructura.

ERRORES FRECUENTES

!

SOLO DIAGRAMAS

Cada diagrama va con texto que explica qué muestra y por qué.

!

SECUENCIAS ≠ CLASES

Cada mensaje debe ser un método del diagrama detallado.

!

JUSTIFICACIONES VACÍAS

"Es más fácil" no basta: nombrar la alternativa y lo que se sacrifica.

!

UN "GESTOR" QUE HACE TODO

Repartir responsabilidades o justificar el control centralizado.

!

COPIAR EL MODELO DEL MUNDO

Agregar clases de la aplicación: interfaz, persistencia, coordinación.

!

DOCUMENTO DESACTUALIZADO

Actualizar los diagramas cada vez que cambie el código.

03

TRABAJO EN CLASE

HOY EN CLASE: ELIJAN SU RUTA

RUTA A

Aún no tenemos documento

1	Pasos 1–2: contexto, alcance y componentes
2	Pasos 3–4: candidatos y responsabilidades
3	Paso 5: 2 o 3 diagramas de secuencia
4	Pasos 6–7: alto nivel y 2 fichas de decisión

RUTA B

Ya tenemos un documento

1	Comparar su documento con las 9 secciones
2	Autoevaluación con la lista de chequeo
3	Coevaluación con otro grupo
4	Corregir lo más débil (casi siempre: decisiones)

LISTA DE CHEQUEO

- [ ] **CONTEXTO**
  Objetivos, no-objetivos, RF, RNF y restricciones
- [ ] **COMPONENTES**
  Diagrama de contexto y componentes con explicación
- [ ] **ROLES**
  Cada clase con estereotipo y descripción
- [ ] **RESPONSABILIDADES**
  Sin huecos ni duplicados; verbos precisos
- [ ] **SECUENCIAS**
  Una por funcionalidad crítica y por qué lo es
- [ ] **CONTROL**
  Estilo de control identificado y justificado
- [ ] **ALTO NIVEL**
  Todas las clases y relaciones, legible
- [ ] **DETALLADO**
  Atributos, métodos con firmas, cardinalidad
- [ ] **DECISIONES**
  3 a 6 fichas con alternativas y trade-offs
- [ ] **CONSISTENCIA**
  Mensajes = métodos; mismos nombres en código

UN BUEN DOCUMENTO RESPONDE

01

¿QUÉ HACE EL SISTEMA?

Secciones 1–2

02

¿CÓMO ESTÁ ORGANIZADO?

Secciones 3, 6 y 7

03

¿CÓMO COLABORAN SUS PARTES?

Secciones 4–5

04

¿POR QUÉ SE DECIDIÓ ASÍ?

Sección 8 · la que más pesa